

修士論文

バードストライク検出に向けた
コンピュータビジョンと
UAV の活用

山形大学大学院
理工学研究科数理科学分野
23321427 千葉歩夢

2025 年 2 月

Abstract

ここに概要を書きます。

Contents

1.1	はじめに	2
1.1.1	背景と目的	2
1.1.2	先行研究の概要	2
1.1.3	引き継ぎの理由と目標	2
1.2	方法とアルゴリズム	3
1.2.1	科学用語の説明	3
1.2.2	検出方法	5
1.2.3	AutoEncoder の改良	6
1.2.4	モデルのアップデート方法	10
1.2.5	プログラミング環境と使用したライブラリ	10
1.2.6	想定環境	12
1.3	使用したデータの説明	12
1.3.1	画像データ	12
1.3.2	モデルデータ	13
1.4	実験結果	13
1.4.1	使用した画像の説明	13
1.4.2	実験手法	14

1.1 はじめに

1.1.1 背景と目的

近年、再生可能エネルギーの 1 つである風力発電の導入が注目されているが、発電機の建設には問題点も多い。バードストライクはその問題点の 1 つであり、回転する風車のブレードに鳥類が衝突する現象である。山形県企業局では、当局が管理している風力発電機に対してバードストライク調査を行っているが、仕事量の多さや正確性に幅があることが懸念されている。この懸念を解消するために、ドローンを用いて発電機周辺の土地を撮影し、自動的に鳥の死骸を発見できるプログラムを作成することが本研究の目的である。

詳しい内容は昨年度の論文で詳細に説明しているので、そちらを参照して欲しい。

1.1.2 先行研究の概要

昨年度、山形大学の先輩である梅津魁秀氏は、深層学習を用いたバードストライク調査プログラムの基礎的な研究を行った。彼の研究では、AutoEncoder を用いて物体検出モデルを作成し、ドローン画像から鳥の死骸を検出する手法を提案した。具体的には、ドローン画像を「物体が存在する画像」と「物体が存在しない画像」に分類し、「物体が存在する画像」と GUI を組み合わせることで手動で鳥の死骸を検出する方法を開発した。彼の研究により、物体画像に関して F2-score が 0.83 という高い精度で物体の有無を判定することができた。

1.1.3 引き継ぎの理由と目標

梅津氏の研究は有望な結果を示したが、実用化にはさらなる改良が必要である。特に、写りゆく四季に適応した汎用的なモデルを構築することと、物体画像から鳥の死骸の画像を自動で検出できるようにすることが挙げられる。本研究では、これらの課題に対処し、より実用的なバードストライク検出システムの開発を目指す。

1.2 方法とアルゴリズム

ここからはバードストライク調査プログラムの作成に使用する方法やアルゴリズムを紹介する。なお、今年度は引継ぎのため、重複する部分は省略し、新規の内容や変更した内容のみ説明する。

1.2.1 科学用語の説明

本研究では、目標達成のために、さまざまな技術を使用している。以下に、研究の中で使用される主要な科学用語を説明する。

AutoEncoder (オートエンコーダ)

AutoEncoder は、入力データを圧縮し、その圧縮されたデータから元のデータを再構築することを目的としたニューラルネットワークの一種である。主にデータの次元削減や特徴抽出に使用される。AutoEncoder はエンコーダとデコーダの 2 つの部分から構成され、エンコーダは入力データを低次元の潜在空間にマッピングし、デコーダはその潜在空間から元のデータを再構築する。

CLIP (Contrastive Language-Image Pre-Training)

CLIP は、OpenAI によって開発されたモデルで、画像とテキストのペアを用いて学習することで、画像とテキストの間の関連性を高精度で捉えることができる。CLIP は、画像エンコーダとテキストエンコーダを用いて、それぞれの入力を高次元のベクトルに変換し、これらのベクトル間のコサイン類似度を最大化するように学習する。これにより、画像とテキストの間の関連性を高精度で捉えることができる。

ResNet (Residual Networks)

ResNet は、深層ニューラルネットワークの一種であり、深いネットワークにおける勾配消失問題を解決するために開発された。ResNet は、残差ブロックを導入することで、非常に深いネットワークでも効果的に学習を行うことができる。これにより、画像認識の精度が大幅に向上することが示されている [1]。

RN50 および RN50x64

RN50 は、ResNet-50 をベースとしたモデルであり、50 層の深層ニューラルネットワークを用いて画像をエンコードする。RN50x64 は、ResNet-50 を 64 倍に拡張したモデルであり、より高精度な画像特徴量を抽出することができる。RN50x64 は RN50 の各層のフィルタ数を 64 倍に増やすことで、より多くの特徴を捉えることができる。これにより、より高精度な画像認識が可能となる。これらのモデルは、CLIP の画像エンコーダとして機能し、テキストエンコーダと組み合わせることで、画像とテキストの関連性を高精度で捉えることができる。

主成分分析 (Principal Component Analysis, PCA)

主成分分析は、高次元データを低次元に変換するための統計手法である。データの分散を最大化する方向に新しい軸（主成分）を見つけ、その軸にデータを投影することで次元削減を行う。これにより、データの重要な特徴を保持しつつ、次元を削減することができる。各主成分の寄与度は、その主成分がデータの分散をどれだけ説明して

いるかを示す指標である。寄与度が高い主成分ほど、データの重要な情報を多く含んでいることを意味する。通常、最初の数個の主成分がデータの大部分の分散を説明するため、次元削減後のデータ解析において重要な役割を果たす。

K-means クラスタリング

K-means クラスタリングは、データを K 個のクラスタに分割するための非階層型クラスタリング手法である。各クラスタは、クラスタの中心（重心）からの距離が最小になるようにデータポイントを割り当てることで形成される。K-means クラスタリングは、データのパターンを見つけるために広く使用される。

コサイン類似度

コサイン類似度は、2 つのベクトル間の類似度を測るための指標である。ベクトル間の角度のコサイン値を計算することで、ベクトルの方向がどれだけ似ているかを評価する。コサイン類似度は、0 から 1 の範囲で値を取り、1 に近いほどベクトルが類似していることを示す。

シード (Seed)

シードは、乱数生成の初期値を設定するための値である。シードを固定することで、実験の再現性を確保することができる。

ゼロショット学習 (Zero-Shot Learning)

ゼロショット学習は、トレーニング時に見たことのないクラスのデータを分類する能力を持つ機械学習手法である。ゼロショット学習では、事前に学習した知識を利用して、新しいクラスのデータを認識する。これにより、少ないデータで新しいクラスを分類することが可能となる。

1.2.2 検出方法

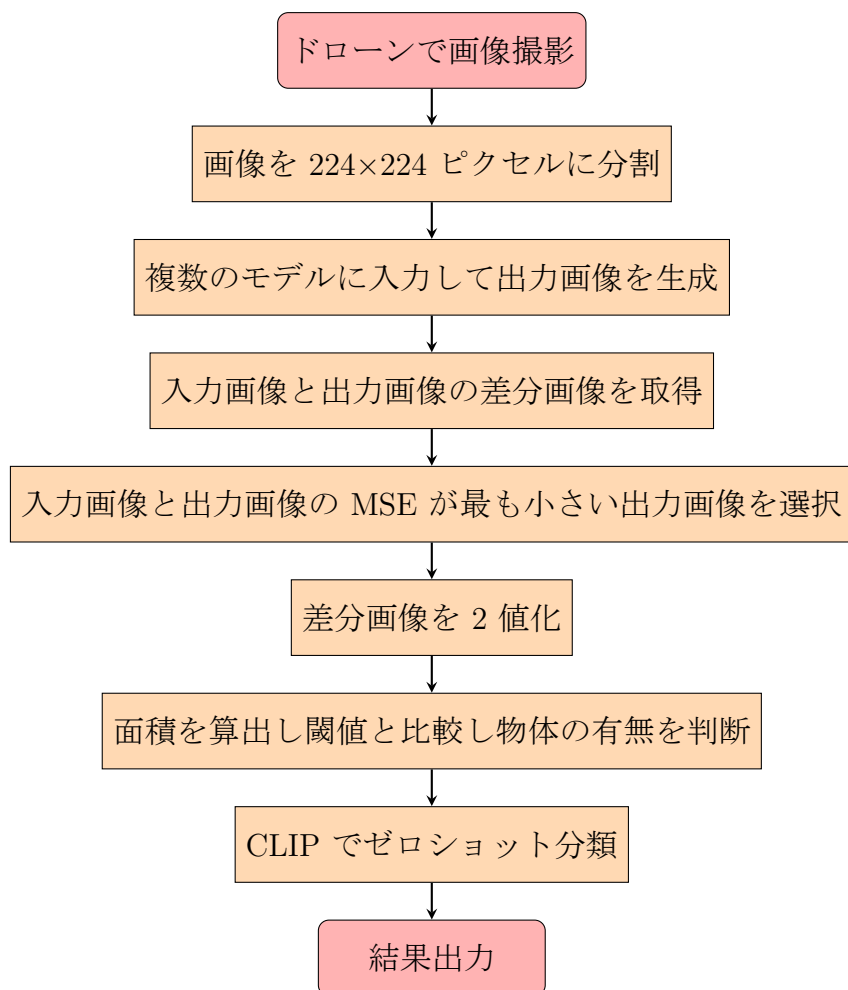


Figure 1.1: バードストライク検出システムの流れ

本研究では、ドローン画像からコンピュータビジョンを用いてバードストライク検出システムを製作した。具体的な流れは以下の通りである。

始めに、ドローンで探索範囲を撮影し、画像（3000×4000 ピクセルが複数枚）を準備する。次に、それらの画像を 224×224 ピクセルに分割し、背景の種類が複数あるため、複数のモデルに入力する。

出力画像と入力画像の差分画像を取ることで、物体部分だけが浮き彫りになった画像を得る。複数のモデルを使用しているため、モデル分の差分画像が生成される。最も適したモデルによる差分画像を選択するために、入力画像と出力画像の平均二乗誤差（MSE）の値が最も小さい画像を採用する。

採用した差分画像に関して、2 値化し、面積を算出して閾値と比較することで物体の有無を判断する。抽出できた物体画像に関しては、その画像が鳥の画像なのかを判定する必要があるが、現状では鳥の死骸の画像が少ないため、教師あり学習は困難である。そこで、ゼロショットで分類できる CLIP の導入を検討している。

ここまでは内部的な処理の話であり、実務での使用のためにさくらの VPS の導入や Django でのアプリケーション開発を学部生二人が行った。これについては巻末に付録として記載する。

1.2.3 AutoEncoder の改良

本研究では、以前の AutoEncoder モデルと比較して、構造をシンプルにすることで性能向上を図った。以下に、以前のモデルと現在のモデルの違い、およびその違いによる改善点について説明する。

モデル用語の説明

- **Conv2d** (入力, 出力): 2 次元畳み込み層。入力チャンネル数と出力チャンネル数を指定し、画像の特徴を抽出するために使用される。例えば、Conv2d (3, 16) は 3 チャンネルの入力画像を 16 チャンネルの出力に変換する。
- **BatchNorm2d**: バッチ正規化層。ミニバッチごとにデータを正規化し、学習の安定性と速度を向上させる。内部共変量シフトを減少させる効果がある。
- **ReLU**: Rectified Linear Unit (ReLU) 活性化関数。入力が 0 以下の場合は 0 を出力し、0 より大きい場合はそのまま出力する。非線形性を導入し、モデルの表現力を高める。
- **MaxPool2d**: 最大プーリング層。指定されたウィンドウサイズ内で最大値を抽出し、空間的なサイズを縮小する。特徴マップのダウンサンプリングに使用される。
- **ConvTranspose2d**: 2 次元逆畳み込み層。畳み込み層の逆操作を行い、特徴マップの空間的なサイズを拡大する。アップサンプリングに使用される。

以前の **AutoEncoder** モデル

以前の AutoEncoder モデルは、以下のような構造を持っていた。

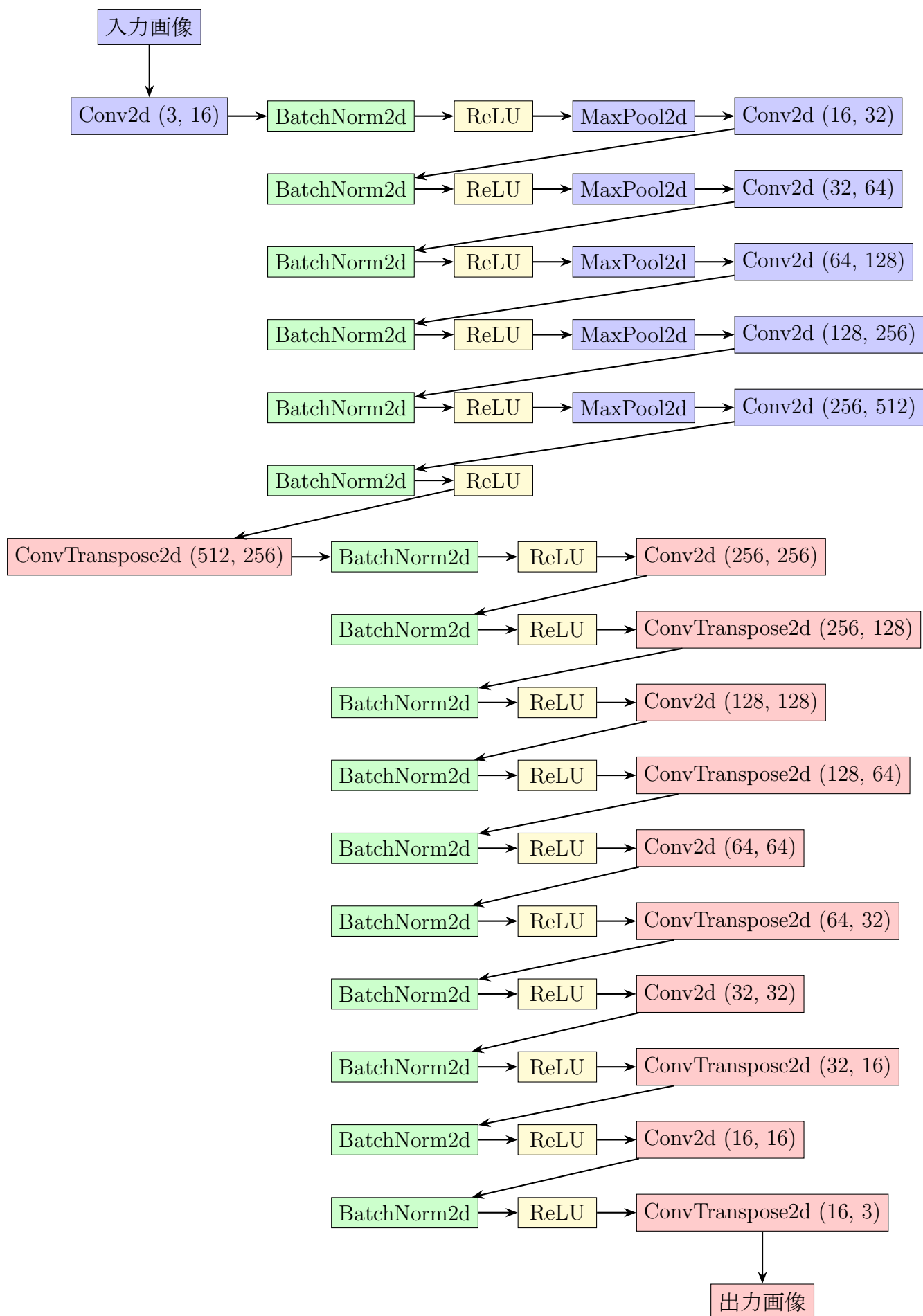


Figure 1.2: AutoEncoder の構造

現在の **AutoEncoder** モデル

現在の AutoEncoder モデルは、以前のモデルと比較して構造がシンプルになっている。以下に、現在のモデルの構造を示す。

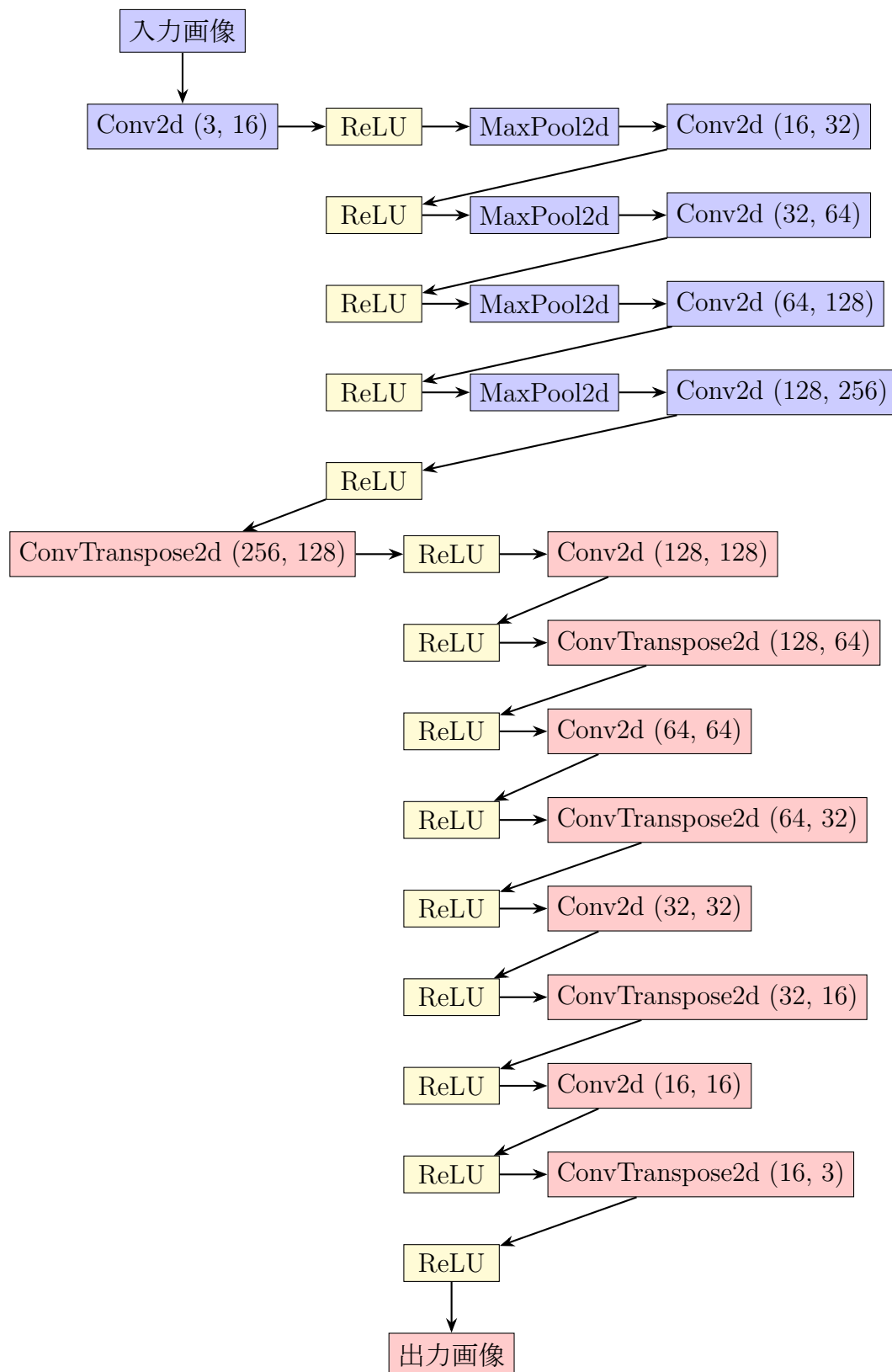


Figure 1.3: 現在の AutoEncoder の構造

変更点

- 畳み込み層と逆畳み込み層の数を減らし、モデルの深さを浅くした。
- バッチ正規化層を削除し、計算コストを削減した。
- ReLU 活性化関数の使用を最小限に抑えた。

改良の効果

これらの変更により、以下のような改善点が見られた。

- 計算コストの削減: モデルの深さを浅くし、バッチ正規化層を削除することで、計算コストが大幅に削減された。これにより、学習速度が向上し、より短時間でモデルのトレーニングが完了するようになった。
- 過学習の防止: モデルのパラメータ数を減らすことで、過学習のリスクが低減された。これにより、テストデータに対する汎化性能が向上し、より安定した結果が得られるようになった。
- モデルの解釈性の向上: 構造をシンプルにすることで、モデルの挙動がより理解しやすくなった。これにより、モデルの改善点や問題点を特定しやすくなり、さらなる改良が容易になった。

以上のように、現在の AutoEncoder モデルは、以前のモデルと比較して構造がシンプルになり、計算コストの削減、過学習の防止、モデルの解釈性の向上といった改善点が見られた。

AutoEncoder モデルのアップデートの重要性

AutoEncoder モデルをアップデートすることは、モデルの性能向上や新しいデータへの適応において非常に重要だ。以下に、モデルをアップデートしないで使い続けることと、アップデートして使うことの差について説明する。

モデルをアップデートしない場合 モデルをアップデートしないで使い続けると、以下のような問題が発生する可能性がある。

- 性能の劣化: 新しいデータや環境に適応できず、モデルの性能が徐々に劣化する可能性がある。例えば、[2] では、モデルの性能がデータの変化に対して敏感であることが示されている。
- 過学習: 古いデータに対して過学習してしまい、新しいデータに対して一般化能力が低下する可能性がある。過学習は、モデルの汎化性能を低下させる主要な要因の一つだ [3]。

モデルをアップデートする場合 一方、モデルを定期的にアップデートすることで、以下のような利点がある。

- 性能向上: 新しいデータや技術を取り入れることで、モデルの性能を向上させることができる。例えば、[1] では、ResNet のような新しいアーキテクチャを導入することで、画像認識の精度が大幅に向上したことが示されている。

- 適応能力の向上: モデルが新しいデータや環境に適応できるようになり、汎化性能が向上する。これは、モデルがより広範なデータセットに対して有効であることを意味する [4]。

1.2.4 モデルのアップデート方法

本研究では、検出に使用した画像を用いてモデルのアップデートを行う。目標は自動アップデートである。アップデート用学習データを作成するためには、以下の二つのステップが必要である。

始めに、全体の画像を背景ごとにクラスタリングする。これは AutoEncoder で特徴抽出を行い、その後主成分分析で次元削減し、k-means クラスタリングを用いることで可能であることがわかった。

次に、画像の中から物体画像を取り除く。これは CLIP を使用することで効果的に行うことができる。CLIP を用いることで、画像内の物体をある程度の精度で検出し、物体が存在しない画像を抽出することができる。

これらのステップに関しては、次の章で詳しく説明する。

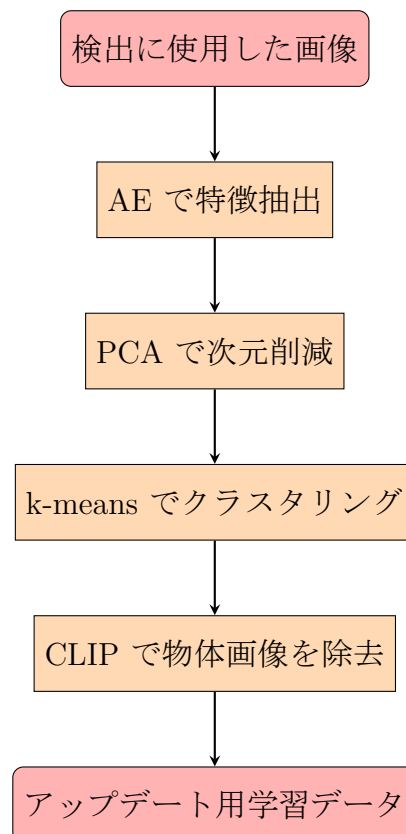


Figure 1.4: モデルのアップデート方法の流れ

1.2.5 プログラミング環境と使用したライブラリ

実験は Python を用いて行った。主なプログラミング環境と使用したライブラリは以下の通りである。

- **Python 3.10.12:** プログラミング言語として使用。
- **collections.defaultdict** (標準ライブラリ): デフォルト値を持つ辞書に使用。

- **csv** (標準ライブラリ): CSV ファイルの操作に使用。
- **cv2** (4.10.0.84): 画像処理に使用。
- **datetime** (標準ライブラリ): 日付と時間の操作に使用。
- **deep_translator** (1.11.4): テキストの翻訳に使用。
- **glob** (標準ライブラリ): ファイルパスのパターンマッチングに使用。
- **matplotlib.pyplot** (3.10.0): グラフの描画に使用。
- **numpy** (1.26.3): 数値計算に使用。
- **open_clip** (2.29.0): CLIP モデルの使用。
- **open_clip.tokenizer** (2.29.0): テキストのトークン化に使用。
- **os** (標準ライブラリ): OS 機能へのアクセスに使用。
- **PIL.Image** (11.0.0): 画像操作に使用。
- **PIL.ImageDraw** (11.0.0): 画像に描画するために使用。
- **PIL.ImageFont** (11.0.0): 画像にフォントを使用するために使用。
- **PIL.ImageOps** (11.0.0): 画像の操作に使用。
- **random** (標準ライブラリ): 乱数生成に使用。
- **re** (標準ライブラリ): 正規表現操作に使用。
- **shutil** (標準ライブラリ): 高レベルのファイル操作に使用。
- **sklearn.cluster.KMeans** (1.6.0): K-means クラスタリングに使用。
- **sklearn.decomposition.PCA** (1.6.0): 主成分分析に使用。
- **sklearn.preprocessing.LabelEncoder** (1.6.0): ラベルエンコーディングに使用。
- **torch** (2.5.1+cu118): 深層学習モデルの構築と訓練に使用。
- **torch.nn** (2.5.1+cu118): ニューラルネットワークの構築に使用。
- **torch.optim** (2.5.1+cu118): 最適化アルゴリズムに使用。
- **torch.utils.data.DataLoader** (2.5.1+cu118): データローダーの操作に使用。
- **torch.utils.data.Dataset** (2.5.1+cu118): データセットの操作に使用。
- **torchvision.transforms** (0.20.1+cu118): 画像の前処理に使用。
- **tqdm** (4.67.1): プログレスバーの表示に使用。

1.2.6 想定環境

```
ルートディレクトリ/  
2023_main.py  
2024_main.py  
AE_clip.ipynb  
AE_deep_check.ipynb  
AE_deep_check_change.ipynb  
AE_deep_finetrain_matomete.py  
AE_deep_main_search.ipynb  
AE_deep_train_matomete.py  
AE_for_newmodel.ipynb  
AE_score_check_labels.ipynb  
imgs/  
models/
```

1.3 使用したデータの説明

1.3.1 画像データ

- **imgs/for_update_img:**
 - 説明: 2024 年 9 月 3 日に撮影した 3000×4000 ピクセルの画像が 156 枚含まれている。
- **imgs/old_all_img/train_img_6048/6048/A_train_img:**
 - 説明: 去年使用した 224×224 ピクセルの画像が背景ごとに分類されずに 1512 枚ずつ、計 6048 枚含まれている。
- **imgs/make_pca_img1000:**
 - 説明: A_train_img と重複しない画像が 1000 枚ずつ、計 4000 枚含まれている。
- **imgs/old_train_img/:**
 - **around_wind/:** jpg が 6048 枚
 - **light_green/:** jpg が 6048 枚
 - **dark_green/:** jpg が 6048 枚
 - **white/:** jpg が 6048 枚
 - 説明: 去年学習に使用した背景ごとに分類された 224×224 ピクセルの画像がそれぞれ含まれている。
- **imgs/test_img:**
 - 説明: 2024 年 9 月 3 日に撮影した for_update_img には含まれていない 3000×4000 ピクセルの画像が 4 枚含まれている。

- **imgs/test_img2/:**
 - **yes/:**
 - * **dark/:** jpg が 100 枚
 - * **light/:** jpg が 100 枚
 - * **white/:** jpg が 100 枚
 - **no/:**
 - * **dark/:** jpg が 100 枚
 - * **light/:** jpg が 100 枚
 - * **white/:** jpg が 100 枚
 - 説明: test_img とは別の 2024 年 9 月 3 日に撮影した for_update_img には含まれていない画像の中から、テスト用に背景ごとの物体が存在する画像と存在しない画像に分類したもの。
- **imgs/update_img_remove/:**
 - **dark_object/:** jpg が 306 枚
 - **dark/:** jpg が 4973 枚
 - **light_object/:** jpg が 1703 枚
 - **light/:** jpg が 7518 枚
 - **white_object/:** jpg が 1925 枚
 - **white/:** jpg が 3064 枚
 - 説明: update_img（今後製作される）を背景ごとに物体が存在する画像としない画像に分けたもの。

1.3.2 モデルデータ

- **models/2023models/:**
 - **AEmodel_dark_green20230927.pth**
 - **AEmodel_light_green20230927.pth**
 - **AEmodel_white20230927.pth**
 - **AEmodel_paved_ground20230927.pth**
 - 説明: 去年製作した背景ごとに 224×224 ピクセルの画像を 6048 枚学習させたモデル。

1.4 実験結果

本章では、モデルのアップデート方法について説明し、実験結果を示す。

1.4.1 使用した画像の説明

本研究では、山形県企業局が提供する風力発電機周辺の画像を使用した。これらの画像は、ドローンを用いて撮影されたものであり、様々な季節や天候条件下で撮影されたものである。画像の解像度は高く、鳥の死骸を検出するための十分な情報が含まれている。

1.4.2 実験手法

実験では、以下の手順でモデルの訓練と評価を行った。

Bibliography

- [1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [4] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521:436–444, 2015.